

Chapter 3

Processes

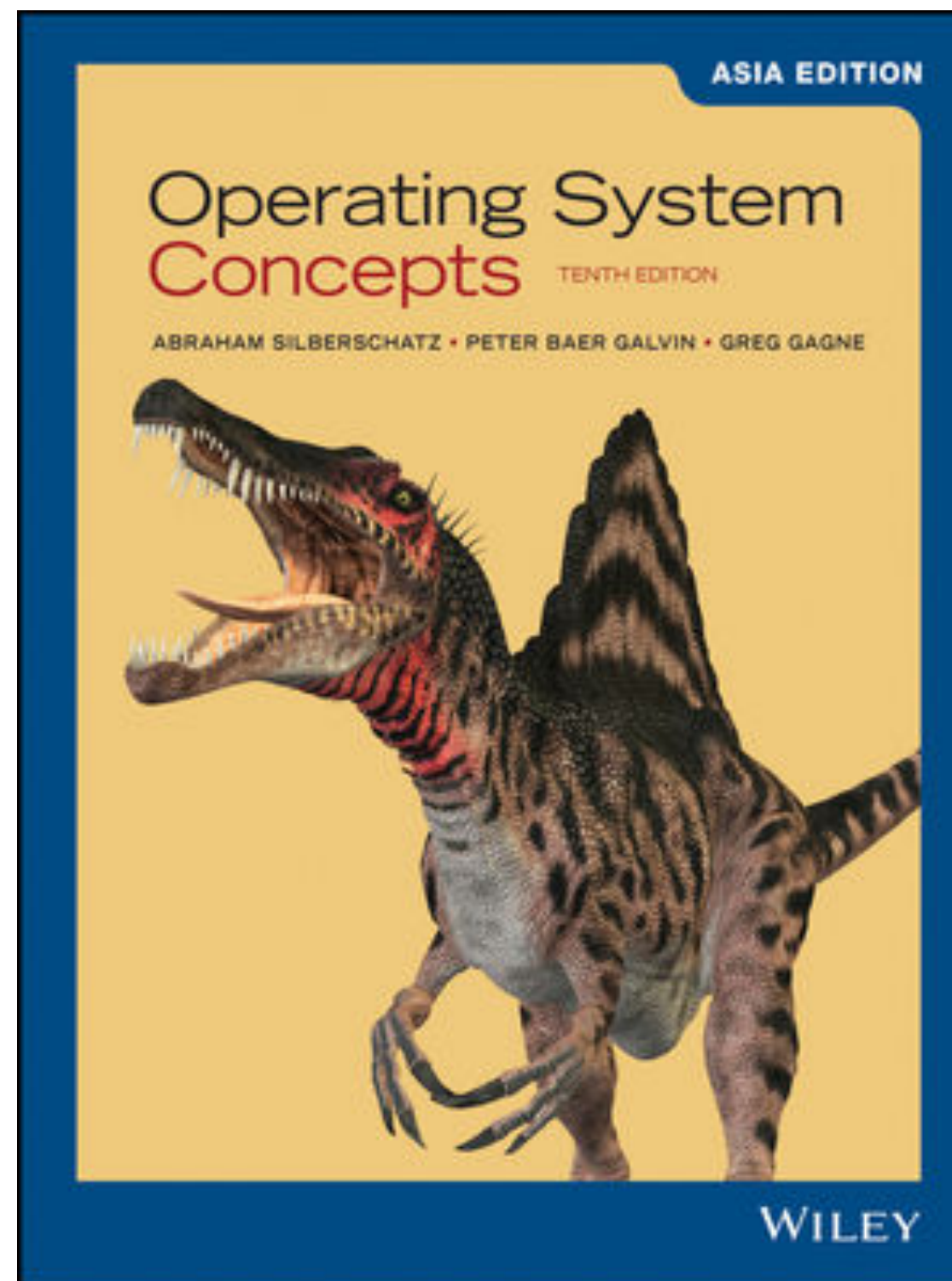
Chih-Yu Lin

National Taiwan Ocean University

Updated date: 2025/9/13

Textbook

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts, Asia Edition," 10/e, ISBN: 978-1-119-58616-6, Wiley



Objectives

- To introduce the notion of a **process** -- **a program in execution**, which forms the basis of all computation
- To describe the various features of processes, including scheduling, creation and termination, and communication
- To explore interprocess communication using **shared memory** and **message passing**
- To describe communication in client-server systems

Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Communication in Client-Server Systems

Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Communication in Client-Server Systems

Process

- An operating system executes a variety of programs:
 - Batch system – **jobs** (批次系統)
 - **Time-shared** systems – **user programs** or **tasks** (類似現在 Time sharing, 但 slice 非常長)
- Textbook uses the terms **job** and **process** almost interchangeably
- Process – **a program in execution**; process execution must progress in sequential fashion
 - The **status** of the current activity of a process is represented by the **value** of the **program counter** and **the contents of the processor's registers**
 - **Context Switch (Process Control Block, PCB)**
PCB 的 save / load

Process

Program: 被动的实体

Process: 主动的

- Program is **passive** entity stored on disk (executable file), process is **active**
 - Program becomes process when executable file **loaded into memory**
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- **One program can be several processes**
 - Consider multiple users executing the same program
 - Server

Layout of a Process in Memory

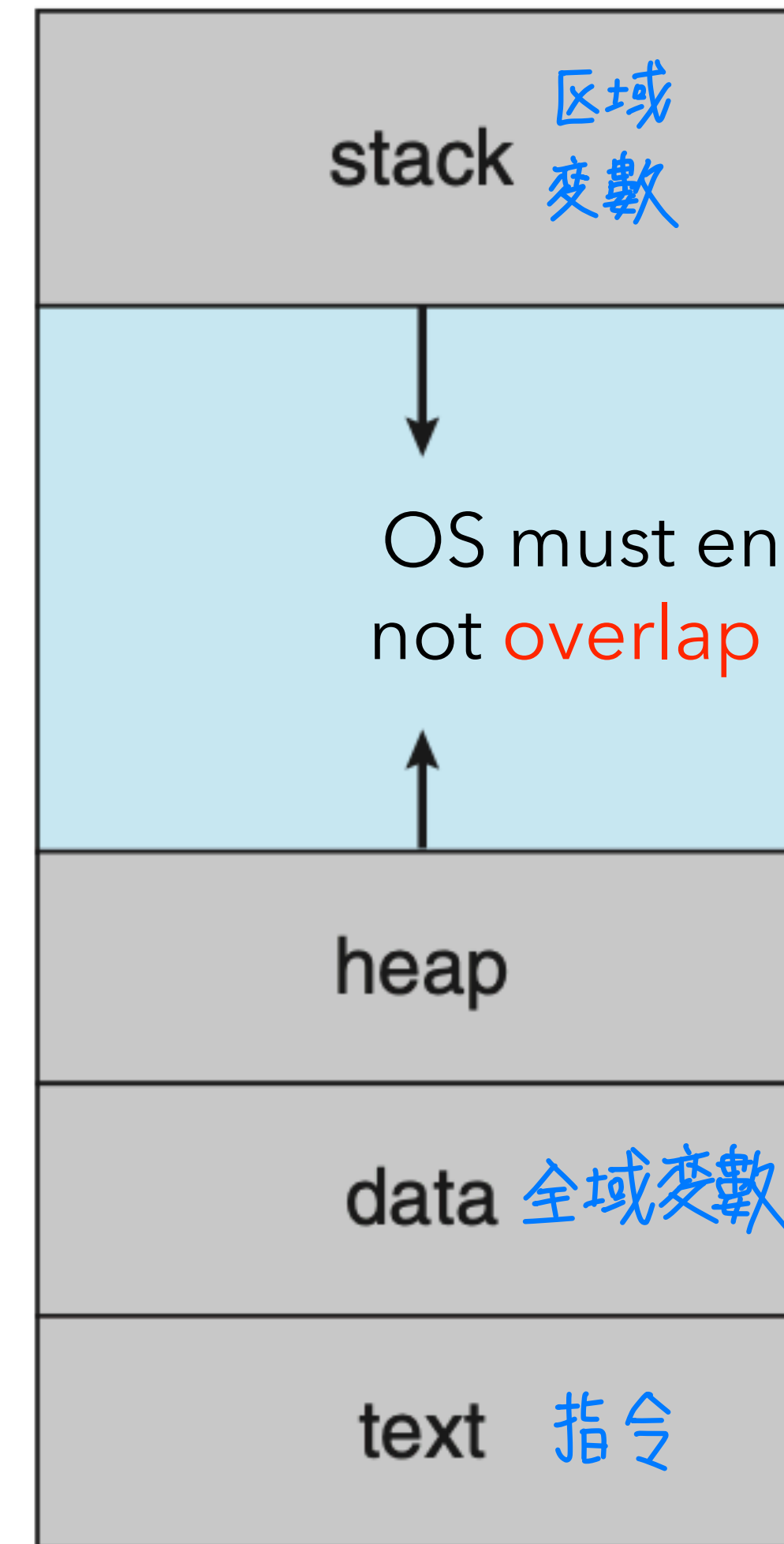
- The memory layout of a process is divided into multiple sections

- **Text section** - the executable code (fixed)
- **Data section** - global variables (fixed)
- **Heap section** - dynamically allocated during program run time
- **Stack section** - temporary data storage when invoking functions (such as function parameters, return addresses, and local variables, **activation record**)
- <https://www.sciencedirect.com/topics/computer-science/activation-record>

Process 佔
記憶體 4 區塊

max

0



區域
變數

馮紐曼架構:

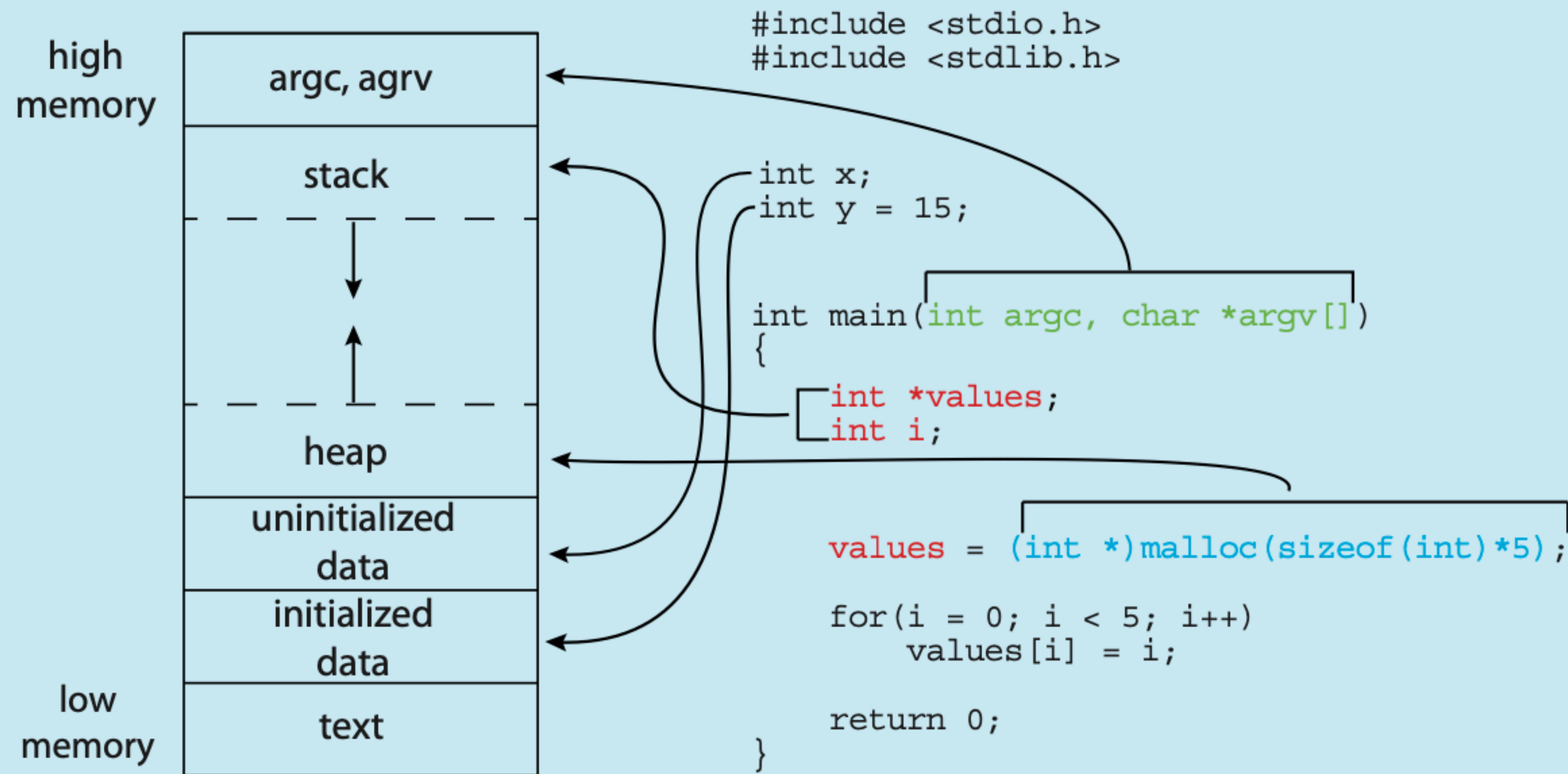
把程式指令記憶體 &
資料記憶體合併成
一大塊記憶體。

全域變數

指令

Memory Layout of a C Program

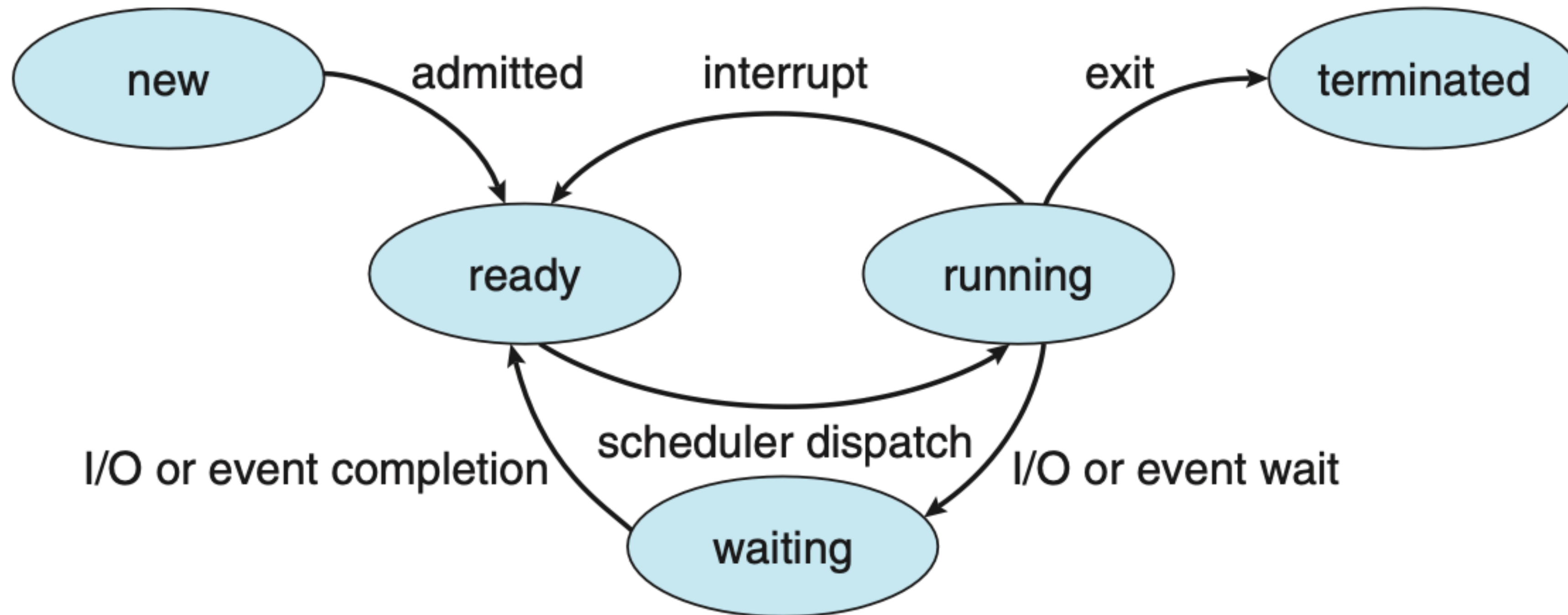
- The global data section is divided into different sections for (a) initialized data and (b) uninitialized data.
- A separate section is provided for the `argc` and `argv` parameters passed to the `main()` function.



Process State

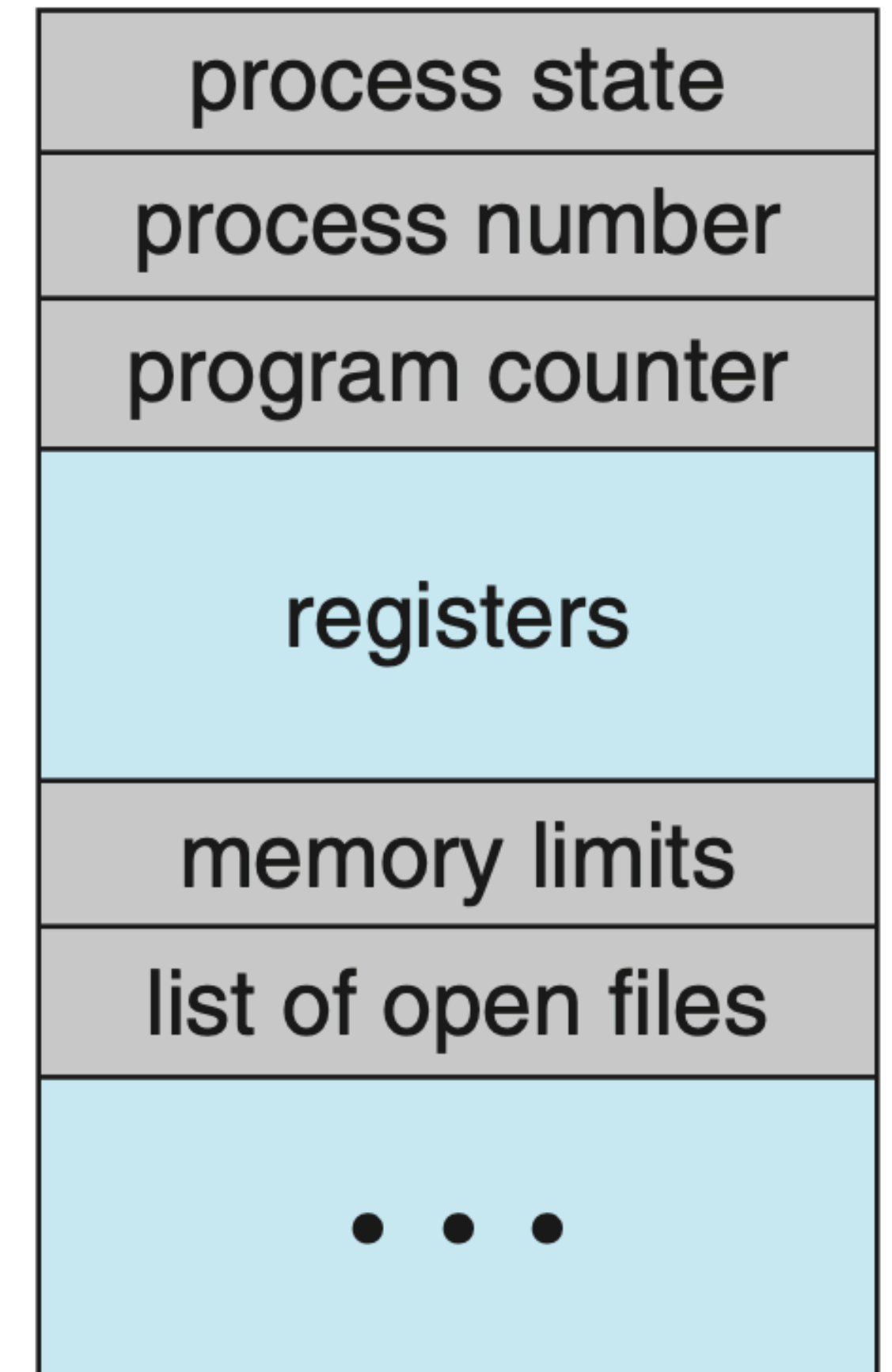
- As a process executes, it changes state
 - **new**: The process is being created
 - **running**: Instructions are being executed (CPU時間正在執行此程式)
 - **waiting**: The process is waiting for some event to occur (讀檔、等I/O)
 - **ready**: The process is waiting to be assigned to a processor
 - **terminated**: The process has finished execution

Diagram of Process State

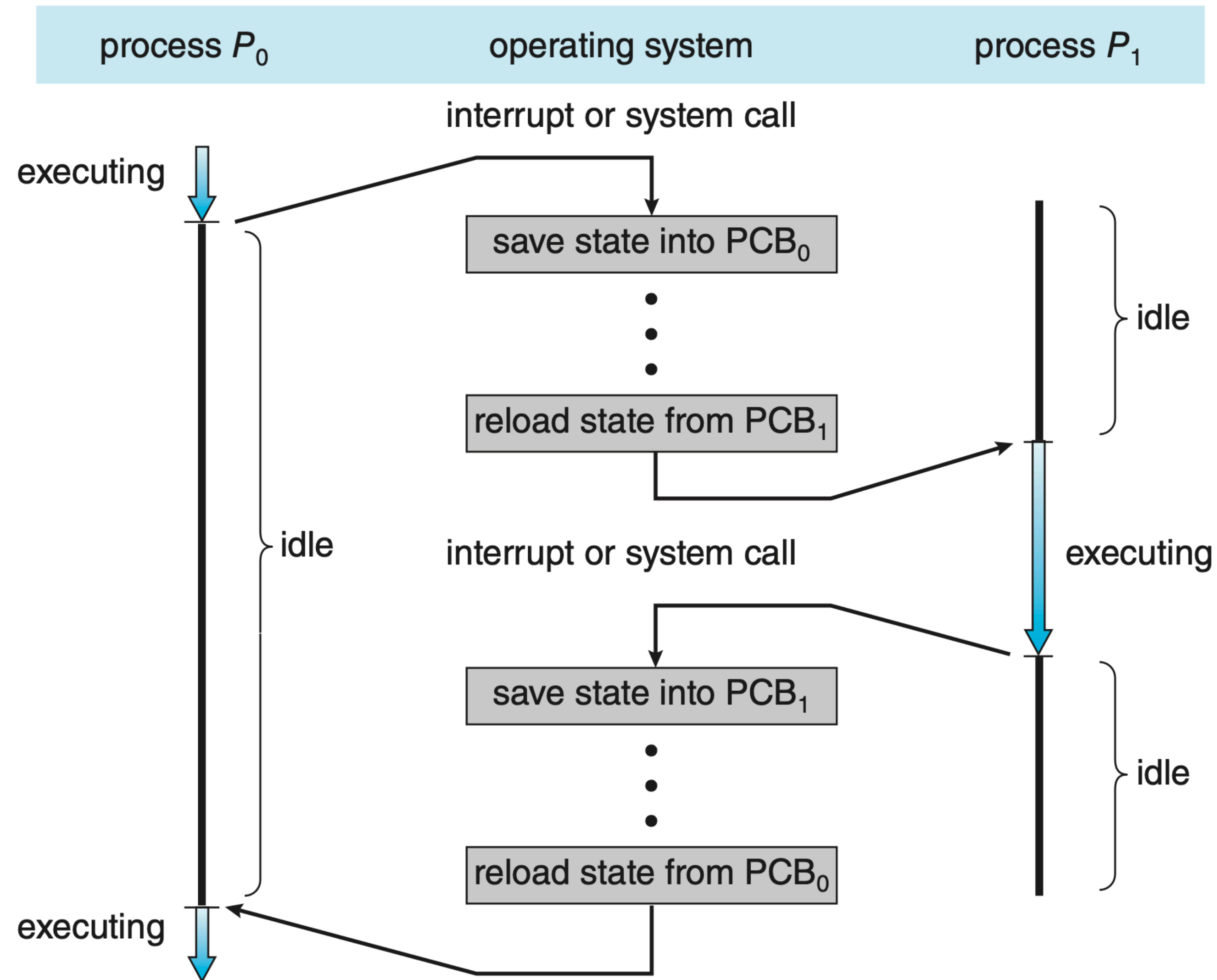


Process Control Block (PCB)

- Information associated with each process (also called **task control block**)
 - **Process state** – running, waiting, etc
 - **Program counter** – location of instruction to next execute
 - **CPU registers** – contents of all process-centric registers
 - **CPU scheduling information** – priorities, scheduling queue pointers
 - **Memory-management information** – memory allocated to the process
 - **Accounting information** – CPU used, clock time elapsed since start, time limits
 - **I/O status information** – I/O devices allocated to process, list of open files
- The PCB simply serves as the repository for **all the data needed to start, or restart, a process**



Context Switch From Process to Process



Threads

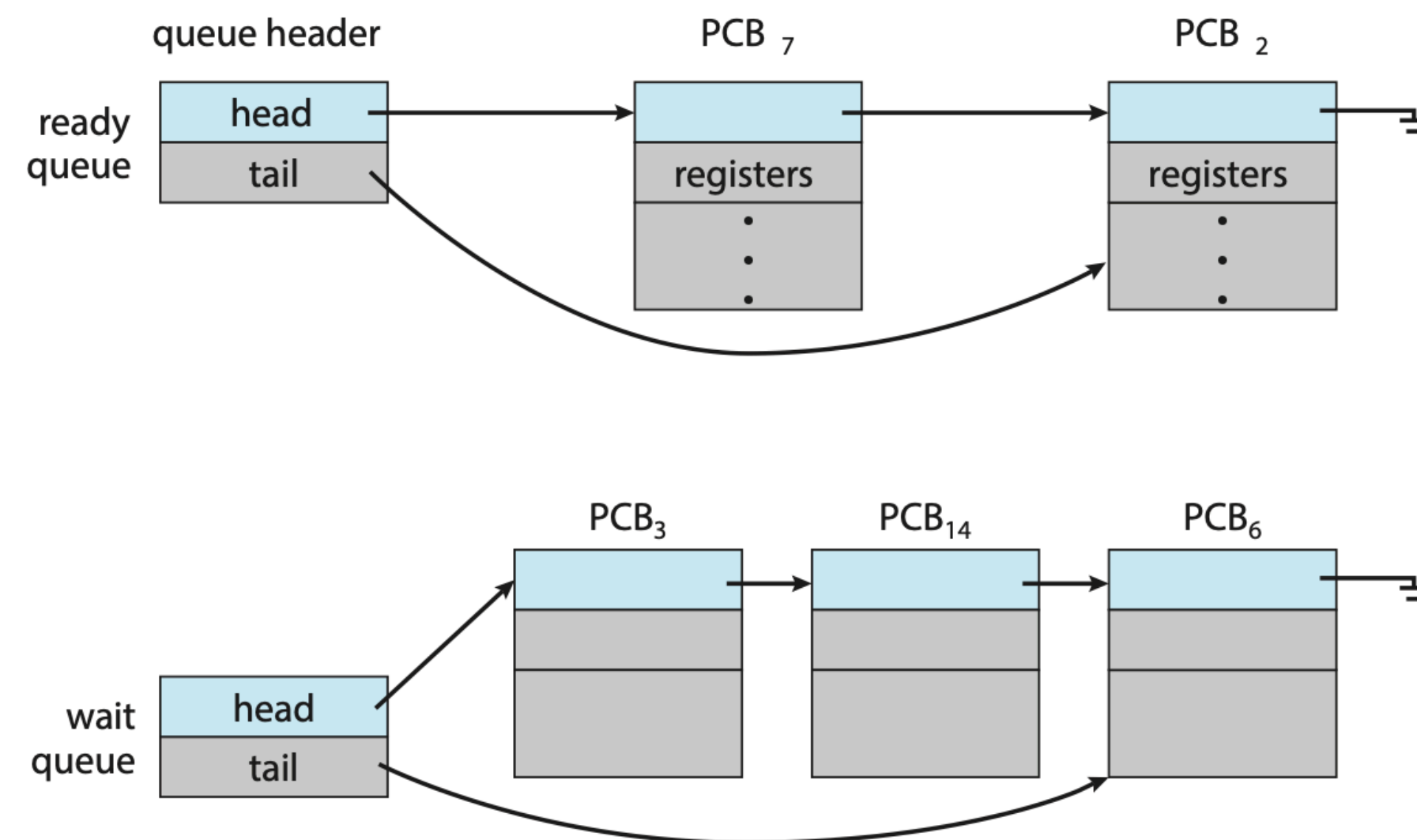
- So far, process has a **single thread** of execution
- Consider having **multiple program counters** per process
- Multiple locations can execute at once
- Multiple threads of control -> threads
- Must then have storage for thread details, multiple program counters in PCB
- See next chapter

Outline

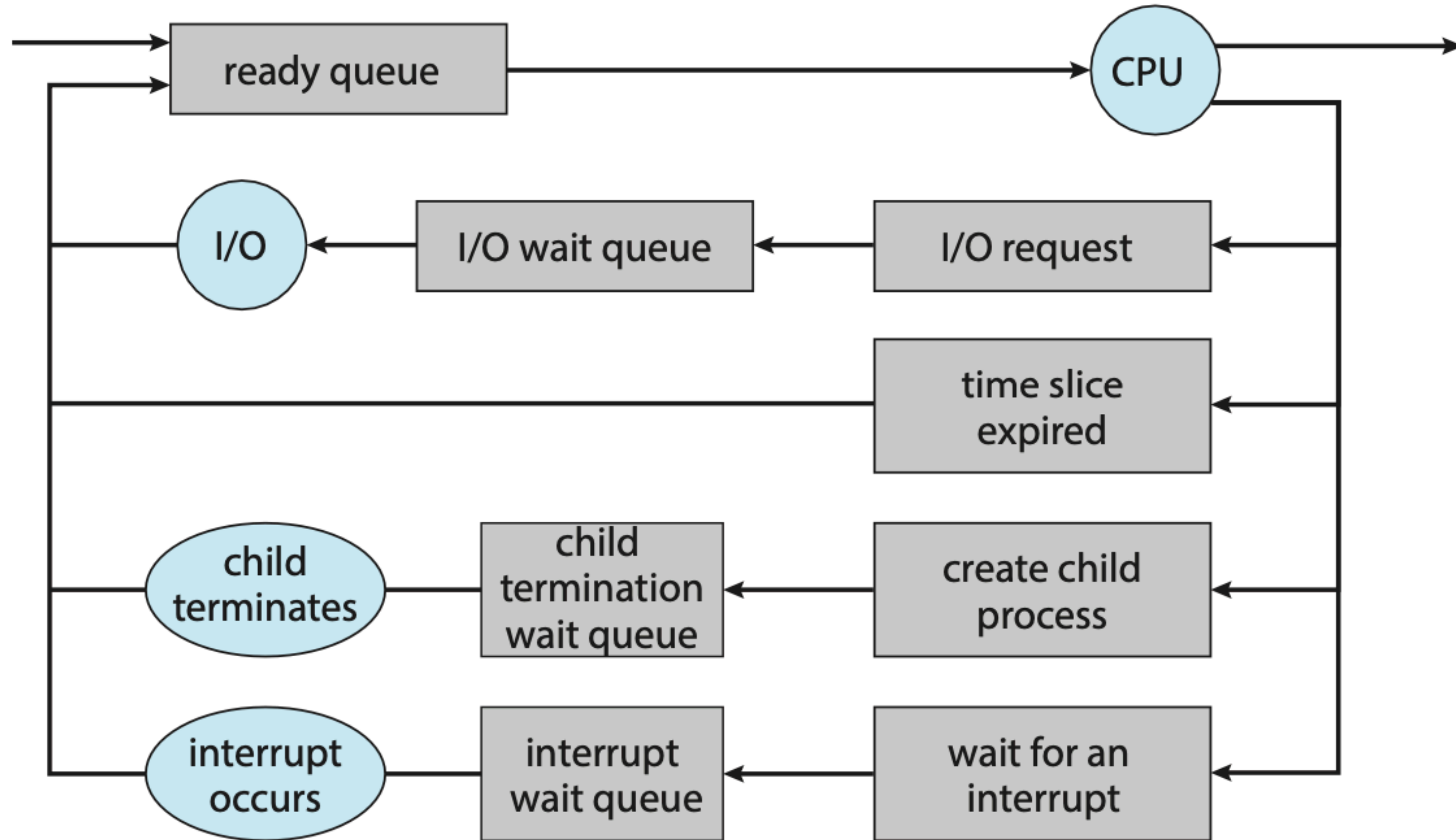
- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Communication in Client-Server Systems

Process Scheduling

- Maximize CPU use, quickly switch processes onto CPU for **time sharing**
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
 - **ready queue** – set of all processes ready and waiting to execute on a CPU's core
 - **wait queues** – set of processes waiting for an I/O device
- Processes migrate among the various queues



Queueing-diagram representation of process scheduling



e.g., Semaphore

Schedulers

- ✧ ● **Short-term** scheduler (or **CPU scheduler**) – selects which process should be executed next and allocates CPU (現多採用的解決方案)
 - Sometimes the **only scheduler in a system**
 - Short-term scheduler is invoked frequently (milliseconds) (**must be fast**) (**time-sharing**)
 - **Modern systems (e.g., Linux, Windows) primarily have only Short-term Schedulers**
- **Long-term** scheduler (or **Job scheduler**) – selects which processes should be brought into the ready queue (早期的批次系統)
 - Long-term scheduler is invoked infrequently (seconds, minutes) (may be slow)
 - The long-term scheduler controls the **degree of multiprogramming** (**The number of processes currently in memory**)
 - Processes can be described as either:
 - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
 - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
 - Long-term scheduler strives for good **process mix**

Context Switch

* Content Switch 对电脑而言
其实是不好的

- When CPU switches to another process, the system must **save** the state of the old process and **load** the saved state for the new process via a context switch
- **Context** of a process represented in the PCB
- Context-switch time is **overhead**; the system does no useful work while switching
- The more complex the OS and the PCB: the longer the context switch
- Time dependent on hardware support
 - Some hardware provides multiple sets of registers per CPU: multiple contexts loaded at once

Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Communication in Client-Server Systems

Operations on Processes

- System must provide mechanisms for:
 - process **creation**
 - process **termination**

Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes

- Generally, process identified and managed via a **process identifier (pid)**
- Resource sharing options

執行緒 識別符

即使執行同個程式, 使用 fork 產生新 process 時,
process 的 Text Section, Data Section, Heap Section 也不同!

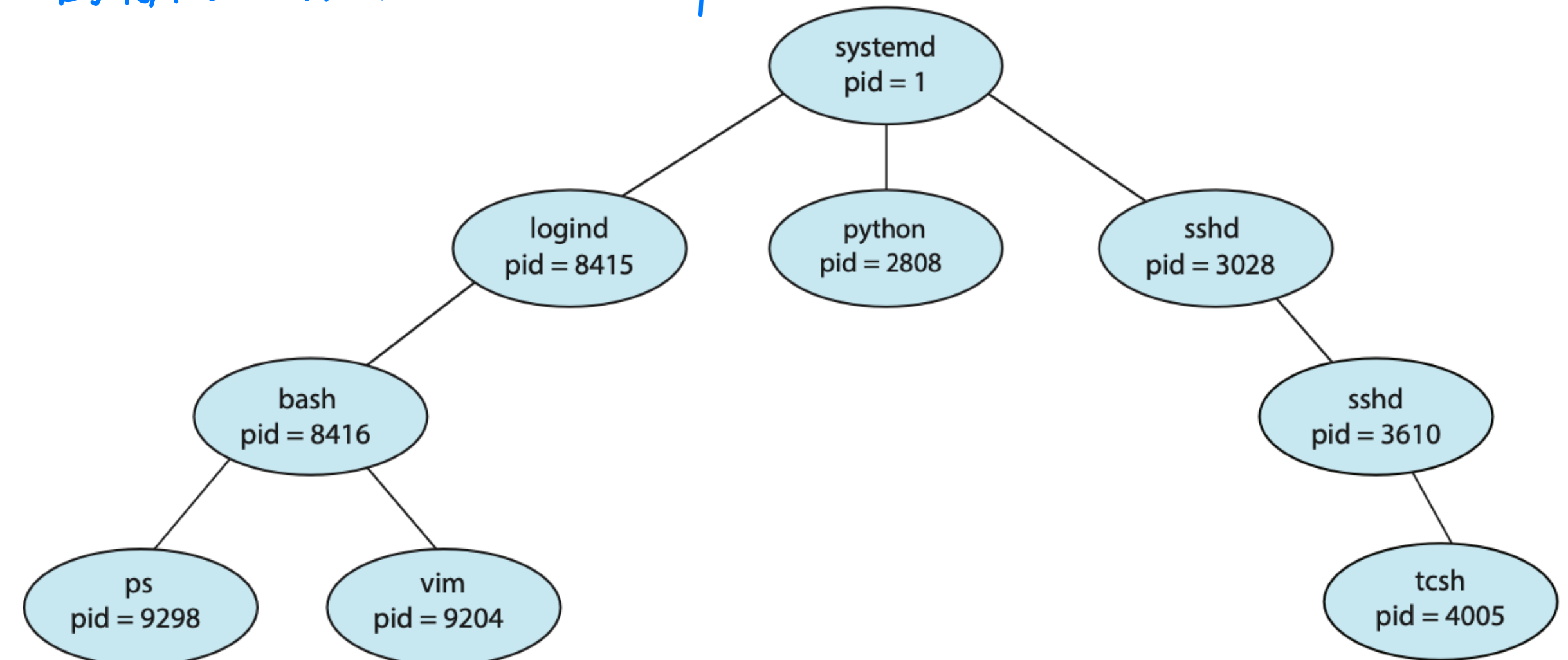
- Parent and children share all resources
- Children share subset of parent's resources
- Parent and child share no resources

- **Execution options**

- Parent and children execute concurrently
- Parent waits until children terminate

- **Address-space options**

- The child process is a duplicate of the parent process (it has the same program and data as the parent).
 - The new process consists of a **copy** of the address space of the original process. (e.g., heap, stack)
- The child process has a new program loaded into it.



Address-space Options

- A **new program** loaded into it
- Reference: <https://burweisnote.blogspot.com/2017/08/execlpexecvp.html>

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main()
{
    pid_t pid;
```

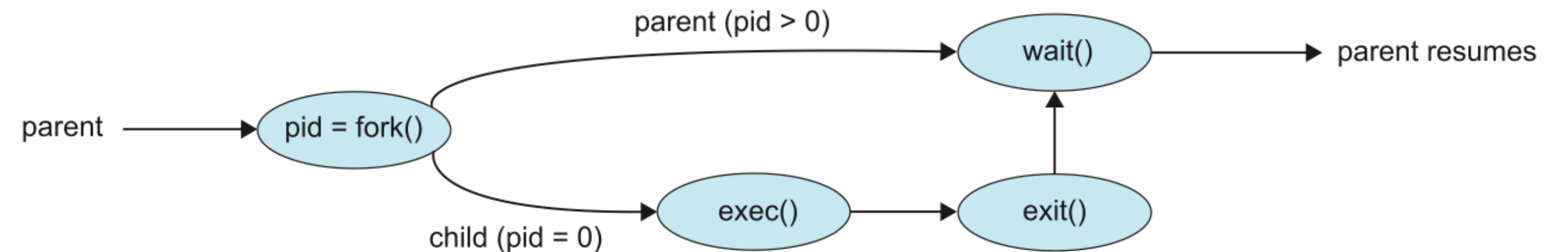
```
    /* fork a child process */
    pid = fork();
```

```
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
```

```
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
```

```
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }
```

```
    return 0;
}
```



```
else if (pid == 0) {
    // Child process
    printf("Child process (PID: %d): Executing hello program...\n", getpid());

    // Execute the "hello" program in the current directory
    execl("./hello", "hello", NULL);

    // If execl returns, an error occurred
    perror("Execution of hello failed");
    exit(EXIT_FAILURE);
}

else {
    // Parent process
    printf("Parent process: Waiting for child (PID: %d) to complete...\n", pid);

    // Wait for the child process to complete
    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid failed");
        exit(EXIT_FAILURE);
    }
}
```

Address-space Options

- Duplicate (Sleep Sort)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/types.h>
5  #include <sys/wait.h>
6
7  // Sleep sort works by creating a separate process for each number
8  // Each process sleeps for a duration proportional to its number
9  // When processes wake up, they print their number in order
10
11 int main(int argc, char *argv[]) {
12     // Check if we have at least one number to sort
13     if (argc < 2) {
14         printf("Usage: %s <num1> <num2> ... <numN>\n", argv[0]);
15         printf("Example: %s 5 3 6 3 1 8 7 2 4\n", argv[0]);
16         return 1;
17     }
18
19     // For monitoring child processes
20     int num_processes = argc - 1;
21     pid_t pid;
22
23     printf("Starting sleep sort for %d numbers...\n", num_processes);
```

```
25     // Create one process for each number
26     for (int i = 1; i < argc; i++) {
27         // Convert argument to integer
28         int num = atoi(argv[i]);
29
30         // Validate input
31         if (num <= 0) {
32             printf("Error: All numbers must be positive integers\n");
33             return 1;
34         }
35
36         // Fork a new process
37         pid = fork();
38
39         if (pid < 0) {
40             // Error in forking
41             perror("Fork failed");
42             return 1;
43         }
```

Address-space Options

- Duplicate (Sleep Sort)

```
44     else if (pid == 0) {
45         // Child process
46
47         // Sleep for a time proportional to the number
48         // Using a multiplier to make the sorting visible but not too slow
49         int sleep_time = num * 100000; // in microseconds (0.1 second per unit)
50         usleep(sleep_time);
51
52         // After sleeping, print the number
53         printf("%d ", num);
54         fflush(stdout); // Ensure output is displayed immediately
55
56         // Exit the child process
57         exit(0);
58     }
59     // Parent process continues to the next iteration
60 }
61
62 // Parent process waits for all child processes to complete
63 for (int i = 0; i < num_processes; i++) {
64     wait(NULL);
65 }
```

→ 時間拉長, 有足夠時間等待, 即使 Context Switch 也不影響。

如果數字變小, 會導致排序結果出錯.....

Context Switch 太快 + 等待時間太短 ⇒ 導致結果出錯

```
Chih-YudeMBP:Slide lincyu$ ./a.out 5 9 8 7 3
Starting sleep sort for 5 numbers...
3 5 7 8 9
Sleep sort completed!
Chih-YudeMBP:Slide lincyu$
```

Practice:

Try adjusting the time on line 49 and see what happens

```
Chih-YudeMBP:Slide lincyu$ ./a.out 5 9 8 7 3
Starting sleep sort for 5 numbers...
3 5 7 8 9
Sleep sort completed!
Chih-YudeMBP:Slide lincyu$ gcc SleepSort.c
Chih-YudeMBP:Slide lincyu$ ./a.out 5 9 8 7 3
Starting sleep sort for 5 numbers...
5 3 7 8 9
Sleep sort completed!
```

num * 1000

num * 100

Process Termination

- Process executes last statement and then asks the operating system to delete it using the `exit()` system call. (Line 57)
child process 可以 call exit() 來結束 Process.
 - Returns status data from child to parent (via `wait()`) (Line 64)
 - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the `abort()` system call. Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting and the operating system does not allow a child to continue if its parent terminates

Process Termination

- Some operating systems do not allow child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
 - **cascading termination:** All children, grandchildren, etc. are terminated.
 - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the wait() system call. The call returns status information and the pid of the terminated process
 - pid = wait(&status);
- **Zombie process:** A process that has finished execution but still remains in the process table because its parent has not yet read its exit status
 - 殭屍進程: Parent 沒有 call wait, 致 Parent 結束時記憶體內還留有其資料。
- **Orphan process:** A process whose parent has terminated while it is still running, causing it to be adopted by the init process (PID = 1) (CascadingTermination.c)
 - 孤單進程 - Parent 進程比 Child 先結束。
 - 若 Parent 先離開, Child 的 Parent 統一變成 PID = 1

Commands for Process Management in Linux

- Track running processes
 - top
 - ps
- Stop a process
 - kill

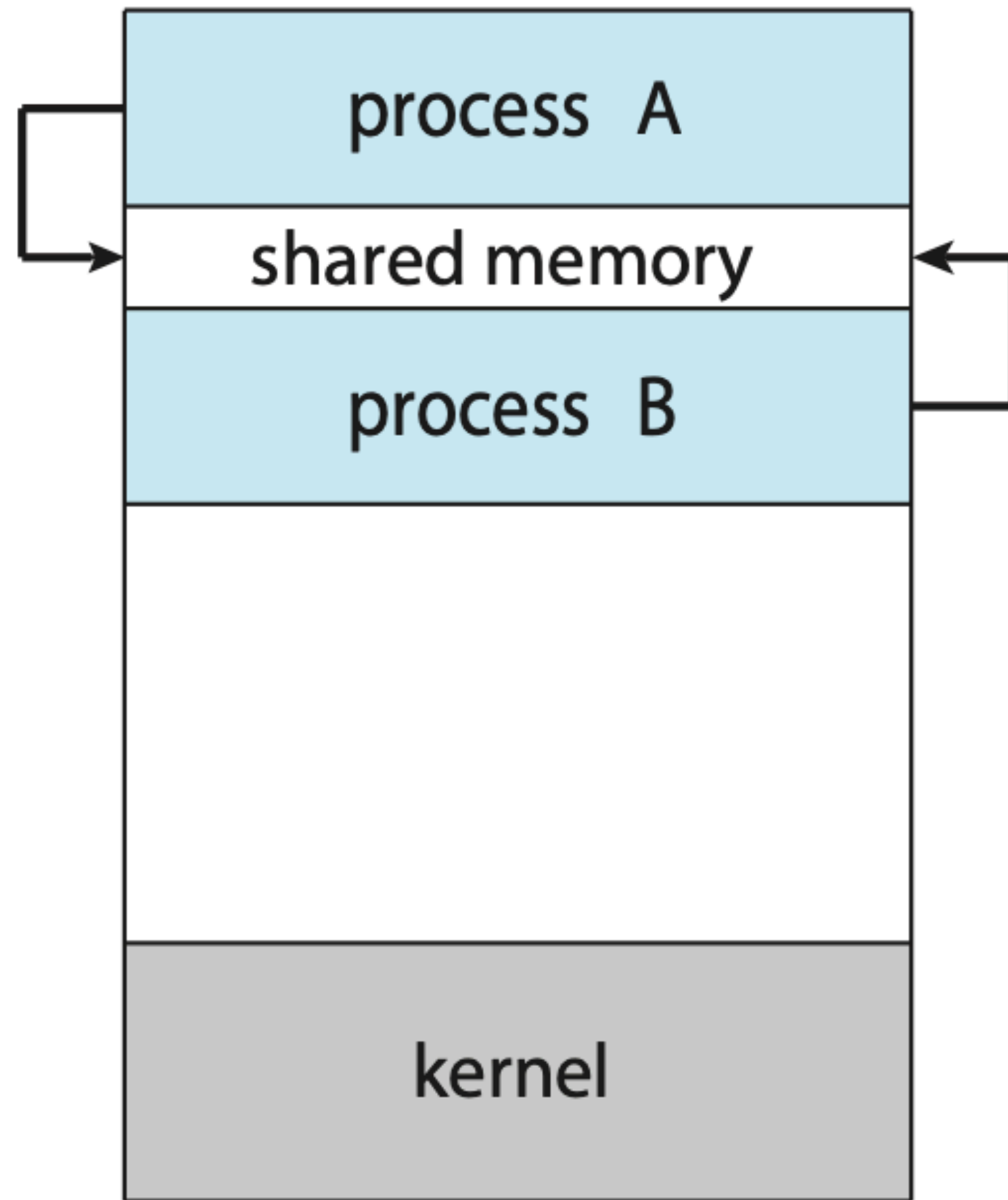
Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Communication in Client-Server Systems

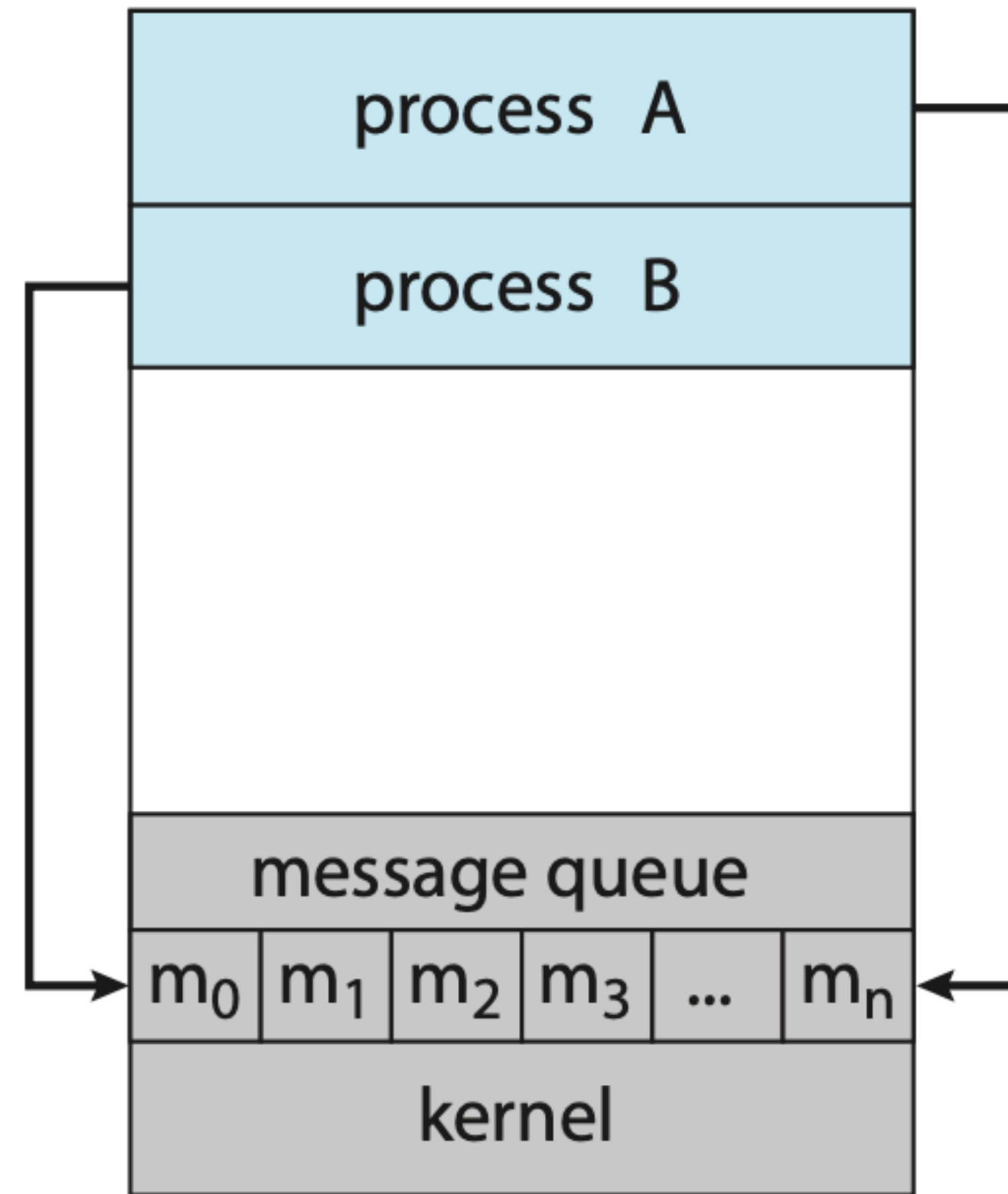
Interprocess Communication

- Processes within a system may be independent or cooperating
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
- Cooperating processes need interprocess communication (IPC)
 - Two models of IPC
 - **Shared memory** (記憶體共享 — 本機端) (e.g. shmat, shmget, ...)
 - **Message passing** (資料轉送 — 網路通訊)

Communications models



(a) Shared memory



(b) Message passing

Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Communication in Client-Server Systems

Shared Memory

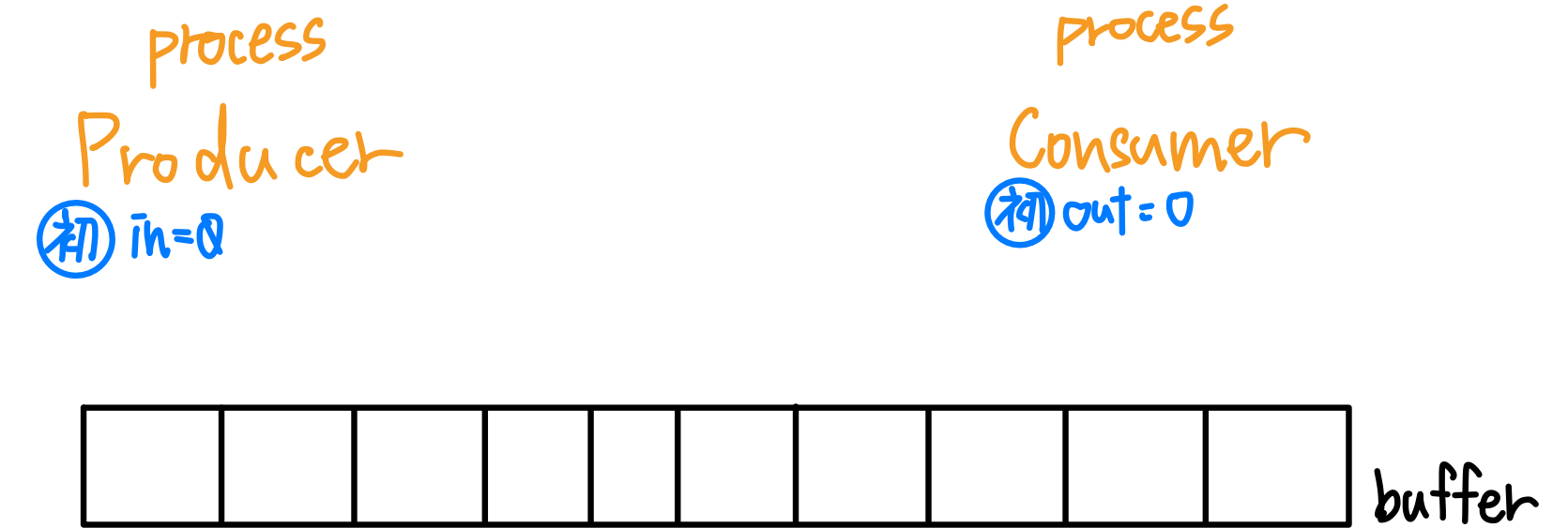
- An area of memory **shared among the processes** that wish to **communicate**
- The communication is under the control of the users processes not the operating system
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory
- **Synchronization** will be discussed in great details in Chapter 6
共享记忆体会引发同步问题

Producer-Consumer Problem

- Bounded Buffer

- The buffer is **empty** when $in == out$

- The buffer is **full** when $((in + 1) \% BUFFER_SIZE) == out$



```
#define BUFFER_SIZE 10
```

```
typedef struct {  
    . . .  
} item;
```

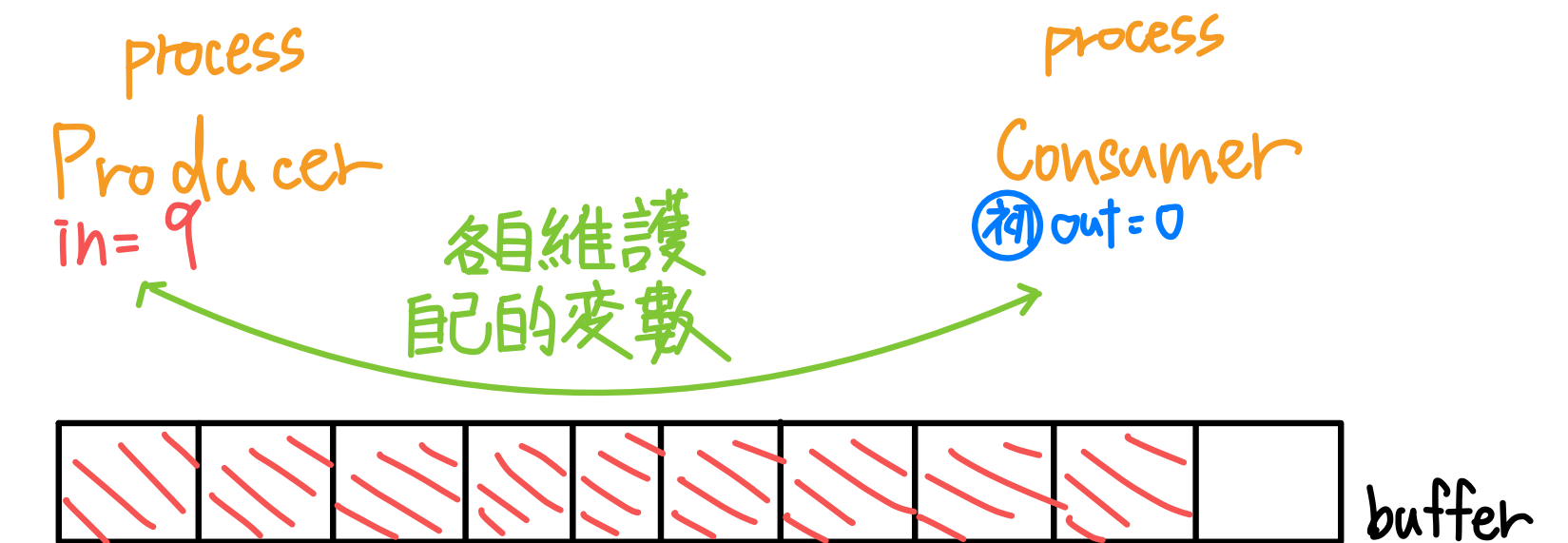
```
item buffer[BUFFER_SIZE];
```

```
int in = 0;    Only modified by the Producer
```

```
int out = 0;   Only modified by the Consumer
```


Producer-Consumer Problem

- The **producer** process using shared memory



```
item next_produced;
```

```
while (true) {
```

```
    /* produce an item in next_produced */
```

$(9 + 1) \% 10 == 0 \dots\dots$

```
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
```

一直往 Buffer 填
直到剩一个空间

```
    buffer[in] = next_produced;
```

```
    in = (in + 1) % BUFFER_SIZE;
```

Can be done **atomically**

```
}
```

需保證這行不能發生

Context Switch

Producer-Consumer Problem

- The **consumer** process using shared memory

```
item next_consumed;

while (true) {
    while (in == out)
        ; /* do nothing */

    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE; Can be done atomically

    /* consume the item in next_consumed */
}
```

Producer-Consumer Problem

- This scheme allows at most **BUFFER SIZE - 1** items in the buffer at the same time.
- Chapter 6: Synchronization

Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Communication in Client-Server Systems

Message Passing

- Mechanism for processes to communicate and to synchronize their actions **without sharing the same address space**
- A message-passing facility provides at least two operations
 - **send**(message)
 - **receive**(message)
- The message size is either fixed or variable

Communication Link

- If processes P and Q wish to communicate, they need to:

- Establish a **communication link** between them
- Exchange messages via send/receive

- Implementation of communication link

- Physical

- Shared memory
- Hardware bus
- Network

- **Logical** 直接傳给对方

- Direct or indirect — 中間有个中間人 (Server)
- Synchronous or asynchronous
- Automatic or explicit buffering

Direct / Indirect

- **Direct Communication**

- Processes must **name** each other **explicitly**
 - send (P, message) – send a message to process P
 - receive(Q, message) – receive a message from process Q

- **Indirect Communication**

- Messages are sent to and received from **mailboxes** (also referred to as **ports**)
 - Each mailbox has a unique id
 - Processes can communicate only if they share a mailbox
 - send(A, message) – send a message to mailbox A
 - receive(A, message) – receive a message from mailbox A

Indirect Communication

- Mailbox sharing
 - P1, P2, and P3 share mailbox A
 - P1, sends; P2 and P3 receive
 - Who gets the message?
- Solutions
 - Allow a link to be associated with at most two processes
 - Allow only one process at a time to execute a receive operation
 - Allow the system to select arbitrarily the receiver
 - Sender is notified who the receiver was

Synchronous / asynchronous

- Message passing may be either **blocking** or **non-blocking**
- Blocking is considered **synchronous** — Blocking: 自然而然達到同步
 - Blocking send -- the sender is blocked until the message is received
 - Blocking receive -- the receiver is blocked until a message is available
- Non-blocking is considered **asynchronous**
 - Non-blocking send -- the sender sends the message and continue
 - Non-blocking receive -- the receiver receives:
 - A valid message, or
 - Null message
- Different combinations possible
 - If both send and receive are blocking, we have a **rendezvous** between the sender and the receiver
 - Synchronous

收到訊息才做下件事

Synchronous / asynchronous

- Producer-consumer becomes trivial

```
Producer message next_produced;

while (true) {
    /* produce an item in next_produced */

    send(next_produced);
}
```

```
Consumer message next_consumed;

while (true) {
    receive(next_consumed);

    /* consume the item in next_consumed */
}
```

Buffering

- **Queue** of messages attached to the link.
- Implemented in one of three ways
 - Zero capacity – no messages are queued on a link.
 - Sender must wait for receiver (rendezvous)
 - Bounded capacity – finite length of n messages
 - Sender must wait if link full
 - Unbounded capacity – infinite length
 - Sender never waits

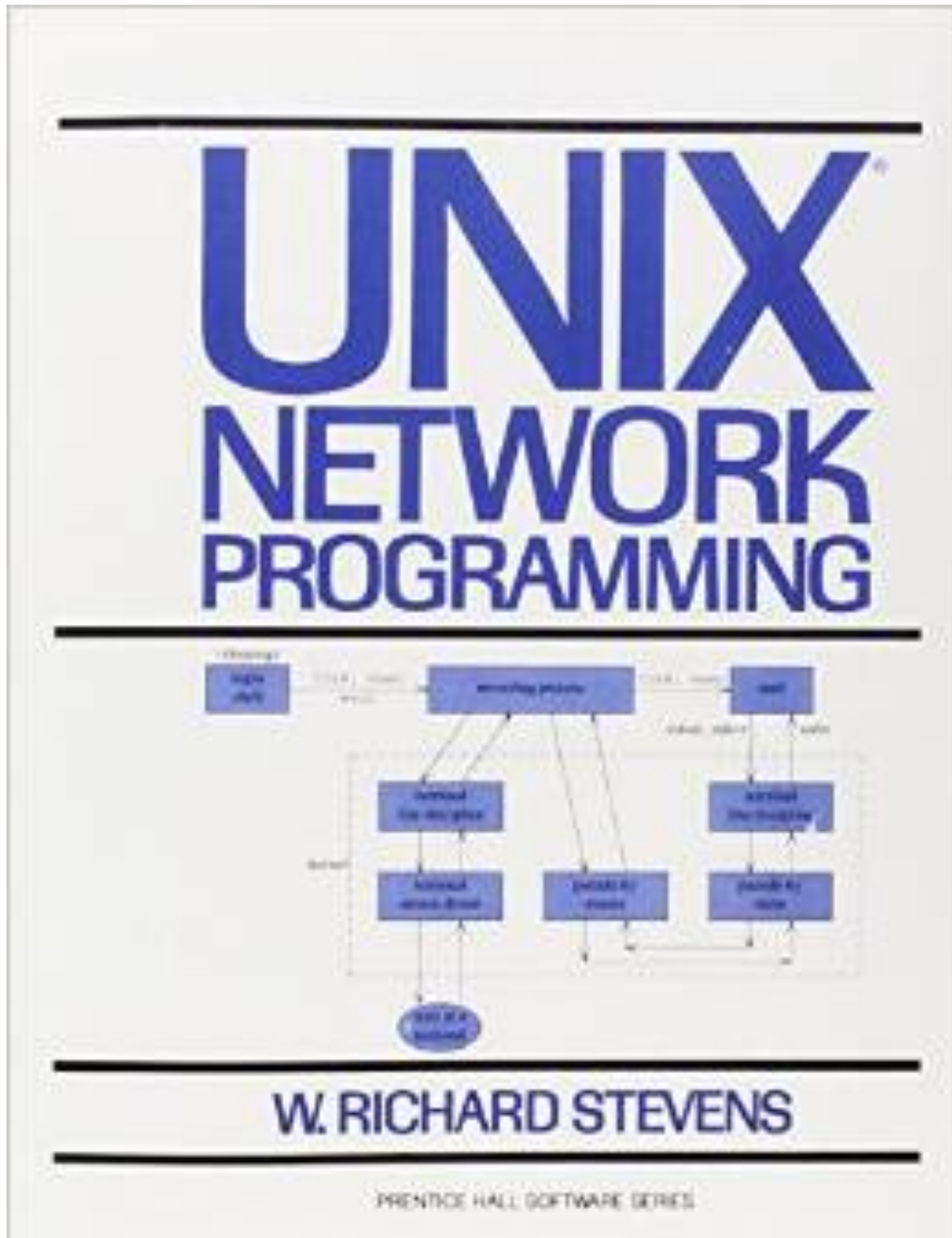
Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Communication in Client-Server Systems

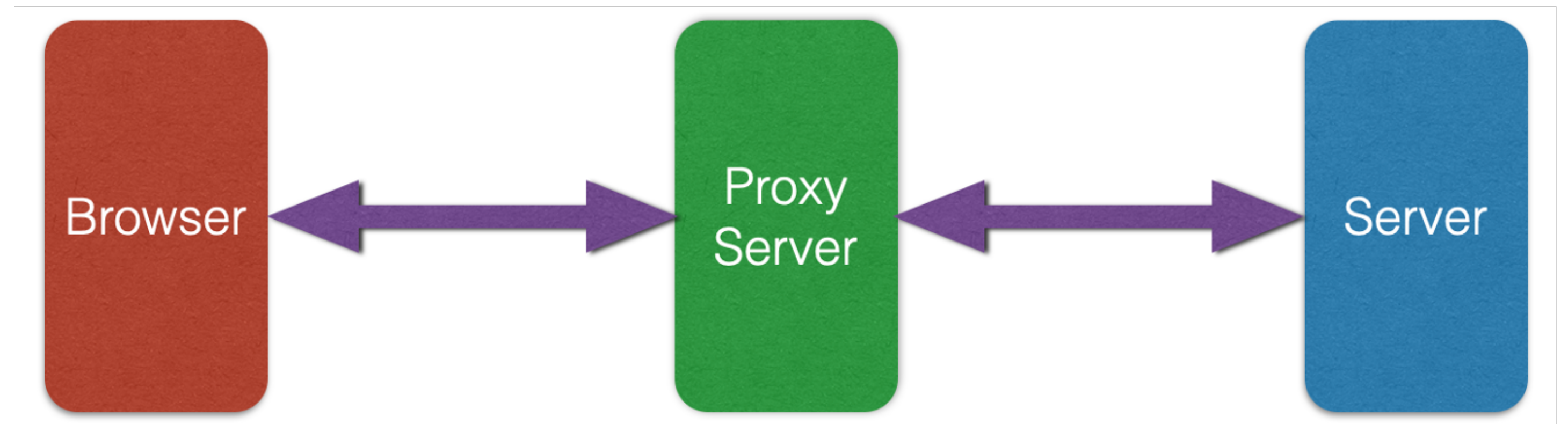
Communications in Client-Server Systems

- Two common forms of client-server communication
 - **Sockets**
 - Sockets allow two processes on different machines to communicate over a network
 - <https://www.geeksforgeeks.org/socket-programming-cc/>
 - **Remote Procedure Calls (RPCs)**
 - RPCs abstract the concept of function (procedure) calls in such a way that a function can be invoked on another process that may reside on a separate computer
 - Reference: <https://aws.amazon.com/tw/compare/the-difference-between-rpc-and-rest/>

Network / Socket Programming



Capstone Project



A single process concurrent server written by C

Reference: <https://gist.github.com/SkrewEverything>